

Lecture 1: Computation and Abstraction

Dr. Changlai Du

Learning goals

After this lecture, you should be able to explain what an algorithm is, distinguish a function from an algorithm, describe the Turing-machine model and the universal Turing machine, state the Church–Turing thesis, compare algorithms through time and space complexity, and explain why hierarchical abstraction makes modern computing systems manageable.

Contents

1	Computation as a Mechanical Process	4
1.1	A motivating problem	4
1.2	Algorithm, input, and output	5
1.3	Correctness and efficiency	6
2	From a Human Calculator to a Turing Machine	7
2.1	The abstract machine	7
2.2	Why one-dimensional and unbounded?	7
2.3	Finite-state control	7
2.4	Unbounded execution and halting	8
2.5	A Turing-machine view of long division	9
3	More Algorithms and More Machines	10
3.1	The Euclidean algorithm	10
3.2	The prime-factorization method	10
3.3	Functions versus algorithms	11
4	The Universal Turing Machine	12
5	The Church–Turing Thesis	13
5.1	Why it is called a thesis	13
5.2	Support for the thesis	13
5.3	Turing completeness	15
5.4	What would challenge the thesis?	15
6	Limits of Computation	16
6.1	Computable numbers	16
6.2	The halting problem	17
6.3	Computation, cognition, and consciousness	18
7	Stored-Program Computers	20
8	Complexity: The Cost of Computation	21
8.1	Input size and asymptotic growth	21
8.2	Searching an attendance list	21
8.3	Fibonacci numbers	22
8.4	P and NP	23

8.5	The complexity zoo	23
9	Hierarchical Abstraction	25
9.1	Leaky abstractions	25
10	Concept Checklist	26
11	Study Questions	27
	Homework	28
	Sources Used for These Notes	29

1 Computation as a Mechanical Process

1.1 A motivating problem

Consider the decision problem

Is 35,500,001 divisible by 113?

Long division gives

$$35,500,001 = 113 \times 314,159 + 34.$$

The remainder is 34, so the answer is *no*.

The familiar long-division procedure consists of a small collection of steps:

1. divide or estimate the next quotient digit;
2. multiply the divisor by that digit;
3. subtract;
4. bring down the next input digit;
5. repeat until no input remains.

Nothing in these steps requires a new creative insight once the procedure has been specified. A person or a machine can follow the same rules. This repeatable, rule-governed character is the first sense in which computation is *mechanical*.

$$\begin{array}{r}
 314159 \\
 113 \overline{) 35500001} \\
 \underline{339} \\
 160 \\
 \underline{113} \\
 470 \\
 \underline{452} \\
 180 \\
 \underline{113} \\
 670 \\
 \underline{565} \\
 1051 \\
 \underline{1017} \\
 34
 \end{array}$$

In treating long division as a mechanical process, we assume that its elementary operations—such as short division, single-digit multiplication, subtraction, comparison, and bringing down the next digit—can themselves be carried out mechanically. We call these primitive operations *instructions*; the finite collection of instructions available to a machine is its *instruction set*. At this level of abstraction, we specify what each instruction does without yet explaining how the machine implements it.

The implementation of such instructions is studied at lower levels of computer systems. Relevant topics and courses include Boolean algebra and switching theory, digital logic design, combinational and sequential circuits, arithmetic circuits such as adders and multipliers, computer

organization and architecture, instruction-set architecture, and processor microarchitecture. These subjects show how elementary mechanical steps can ultimately be realized using logic gates, registers, data paths, and control circuits.

1.2 Algorithm, input, and output

Algorithm

An *algorithm* is a finite, unambiguous sequence of instructions for solving a class of problems. It specifies how to transform an input into an output.

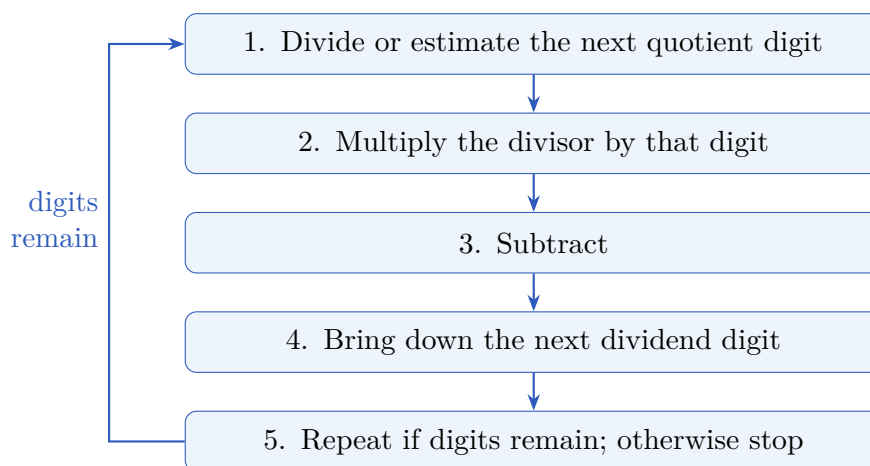


Figure 1: The five-step long-division algorithm.

For the divisibility example, an input may consist of a dividend a and a divisor b . Although our numerical example is small enough to write on one line, computer science studies the entire family of possible inputs rather than only this one instance. The dividend may be extremely long—possibly millions or billions of digits—so an algorithm must describe a uniform procedure that works regardless of the particular input length. If the dividend has n digits, we treat n as its input size and ask how the algorithm’s time and memory usage grow as n increases. The requested output then determines the type of problem:

Problem type	Required output	Example
Decision	yes or no	Is a divisible by b ?
Search	a particular object or value	What is the remainder of a/b ?
Counting	the number of valid objects	How many divisors does a have?
Generation	a sequence or structured output	Generate successive digits of π
Optimization	the best feasible object or value	Which valid solution has minimum cost?

A search problem asks us to produce a particular object, often called a *solution* or *witness*, that satisfies specified conditions. For many important search problems, if we are given a candidate solution, we can verify efficiently that it is correct, even though finding such a solution may be much harder. This distinction between finding and verifying a solution will become important when we discuss the complexity class NP.

The output 34 solves the remainder-search problem. The output *no* solves the original decision problem.

1.3 Correctness and efficiency

The same function can have several algorithms. Repeatedly subtracting 113 from 35,500,001 will eventually produce the correct remainder, but it needs roughly 314,159 subtractions. Long division processes the input one digit at a time and therefore requires a number of rounds proportional to the number of digits.

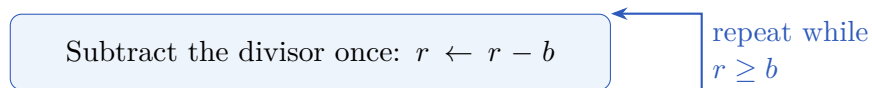


Figure 2: The repeated-subtraction algorithm for the divisibility problem.

This example separates two questions:

- **Computability:** Can any mechanical procedure solve the problem for every valid input?
- **Complexity:** How much time and memory does a particular procedure require as the input grows?

2 From a Human Calculator to a Turing Machine

When performing long division by hand, we use paper as memory, symbols for data, eyes to read, a hand and pencil to write, a finite collection of remembered instructions, and rules that determine the next action. Turing's model keeps only these essential components.

2.1 The abstract machine

A Turing machine has:

- an unbounded one-dimensional tape divided into cells;
- a finite tape alphabet, including a blank symbol;
- a read/write head that scans one cell at a time;
- a finite set of control states, including a start state and one or more halting states;
- a transition function that determines the next action.

The transition function has the form

$$\delta(q, a) = (q', b, D),$$

where q is the current state, a is the scanned symbol, q' is the next state, b is the symbol written to the tape, and $D \in \{L, R\}$ is the direction of movement.

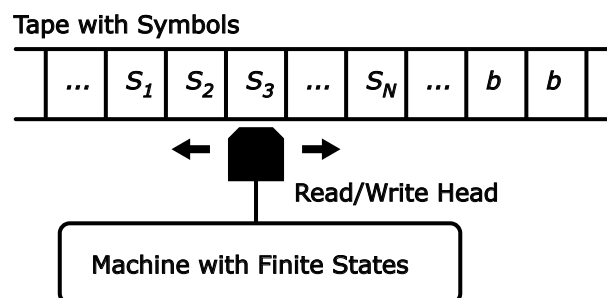


Figure 3: The basic components of a Turing machine.

2.2 Why one-dimensional and unbounded?

A sheet of paper is two-dimensional, but its cells can be enumerated and placed along a one-dimensional tape. This transformation simplifies the mechanism: the head needs only to move left or right. The precise enumeration is unimportant as long as it preserves all information.

The tape is *unbounded* because the size of a valid input has no fixed upper limit. For any particular run, the machine uses only finitely many cells. If it needs more, the model permits more tape. This idealization separates computability from the accidental memory limit of a physical computer.

2.3 Finite-state control

The machine has only *finitely many* states. This resembles human mechanical work: the brain keeps limited control information in mind, while details are recorded externally on paper. In

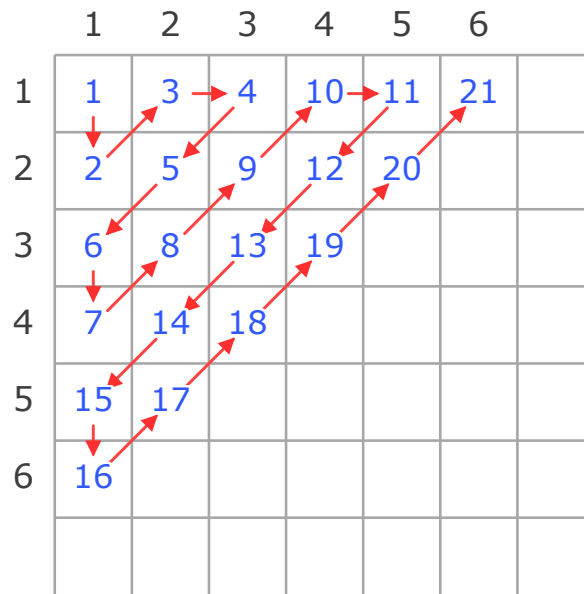


Figure 4: Mapping a two-dimensional grid to a one-dimensional tape.

a Turing machine, the states provide limited internal control and the tape provides external memory.

LEGO offers a similar analogy. There are finitely many types of pieces, yet arbitrarily many copies can be combined to build models without a fixed size limit. Likewise, a Turing machine reuses finitely many states and rules across arbitrarily many tape cells.

Finite control is required for two reasons. First, a mechanical controller must have a finite description and use limited physical resources. Second, infinitely many states could store one answer for every possible input. The machine would then be an infinite lookup table or enumeration, not an algorithm applying reusable steps. Finite states prevent this shortcut and require genuine computation by a finite set of rules.

2.4 Unbounded execution and halting

Unbounded execution means that the model imposes no fixed upper limit on the number of steps a computation may take. It does not mean that every computation runs forever. A particular run may halt after finitely many steps, but different inputs may require increasingly many steps, and some runs may never halt.

Unbounded execution is different from unbounded memory. A machine can execute forever using only a fixed amount of tape by repeatedly visiting the same cells in a loop. Conversely, the unbounded tape means that no fixed memory limit is imposed in advance, although only finitely many cells have been visited at any finite point in a run. Time counts transitions; memory counts the tape cells used.

Halting

A computation *halts* when the machine reaches a designated halting state or when no transition rule applies. This event is its *halting condition*. If the condition is never reached, the machine continues executing indefinitely.

For algorithms computing a function, a correct algorithm must halt on every input covered by

its specification and produce the required output. Reactive systems such as network services, by contrast, are specified to run indefinitely; their correctness is defined by safety and liveness properties rather than eventual termination.

2.5 A Turing-machine view of long division

Fix the divisor at 113. A state can record the current remainder r , where $0 \leq r < 113$. When the head reads the next decimal digit d , compute

$$t = 10r + d, \quad c = \left\lfloor \frac{t}{113} \right\rfloor, \quad r' = t \bmod 113.$$

The machine writes the quotient digit c to the tape, moves right, and changes to the state representing r' . The remainder can be kept in the finite control because it is always one of the 113 values from 0 to 112. The quotient is different: it can have nearly as many digits as the dividend and therefore has no fixed maximum length. A finite set of states cannot store an arbitrarily long quotient, so the machine must write its digits incrementally to the tape. For the input 35500001, the computation is:

Digit read	Quotient prefix	Remainder state
3	0	3
5	00	35
5	003	16
0	0031	47
0	00314	18
0	003141	67
0	0031415	105
1	00314159	34

After the machine reaches the blank cell, it halts. The final non-leading-zero quotient is 314159 and the remainder state is 34.

What the model establishes

The Turing machine is not a blueprint for a fast computer. It is a deliberately minimal mathematical model that makes the phrase “mechanical procedure” precise enough to study.

3 More Algorithms and More Machines

3.1 The Euclidean algorithm

Another example problem is to find the *greatest common divisor*: the largest positive integer that divides both given integers without a remainder. In this example, we want to find the greatest common divisor of 404 and 96, written $\text{gcd}(404, 96)$.

To compute it, repeatedly replace the larger pair by the divisor and the remainder:

$$\begin{aligned} 404 &= 4 \cdot 96 + 20, \\ 96 &= 4 \cdot 20 + 16, \\ 20 &= 1 \cdot 16 + 4, \\ 16 &= 4 \cdot 4 + 0. \end{aligned}$$

The last nonzero remainder is 4, so $\text{gcd}(404, 96) = 4$.

If a and b have many digits, finding the quotient and remainder in each round is not a single elementary instruction: it can require long division. Thus, Euclid's algorithm uses division as a subroutine. The number of rounds and the work needed for each division are separate parts of its running cost.

Like long division, this procedure uses a fixed collection of operations and a repeat rule. We can therefore design another Turing machine whose states implement it.

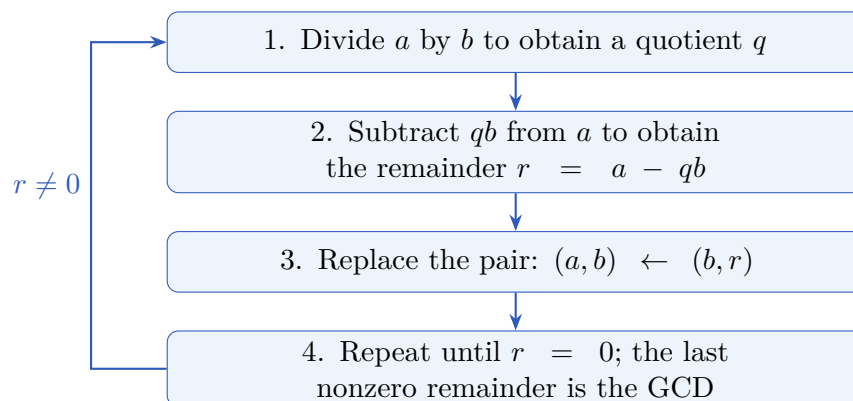


Figure 5: The four-step Euclidean algorithm for computing a greatest common divisor.

The Euclidean algorithm can be implemented directly as follows:

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

3.2 The prime-factorization method

Prime factorization provides another method for finding a greatest common divisor. Factor each input into primes, retain every prime shared by both factorizations, and use the smaller exponent when a shared prime occurs more than once. For the same example,

$$404 = 2^2 \cdot 101, \quad 96 = 2^5 \cdot 3.$$

The only shared prime is 2, and the smaller exponent is 2. Therefore,

$$\gcd(404, 96) = 2^2 = 4.$$

This method computes the same GCD function as the Euclidean algorithm, but it requires prime factorizations and therefore has a very different computational cost.

Homework

Our course code is 44,511,105. Find its prime factorization.

The homework is intended to make the cost of an algorithm tangible: finding prime factors can take much more work than checking a proposed factor by division. Testing whether a remaining large factor is prime is a separate algorithmic task. For very large integers, the difficulty of finding factors also motivates public-key cryptography, including RSA. This is a question of *efficiency*, not computability: factoring is possible by a mechanical procedure even if a particular instance is hard to solve in practice.

3.3 Functions versus algorithms

Function and algorithm

The *problem specification* states what output is required for each valid input. When every input has a unique output, this input–output relationship is a *function*; a search problem may instead permit several valid outputs. An *algorithm* is a particular sequence of instructions for computing an output that satisfies the specification. Thus, a function specifies *what* must be computed, whereas an algorithm specifies *how* to compute it. Several different algorithms may compute the same function or solve the same problem.

For example, $\gcd(a, b)$ is a function. The Euclidean algorithm and a factorization-based method are algorithms that compute it.

Integer multiplication is also a function. The schoolbook multiplication algorithm for two n -digit integers uses $O(n^2)$ single-digit multiplications, while faster multiplication algorithms compute the same function with a lower asymptotic running time.

4 The Universal Turing Machine

Building a separate physical machine for every algorithm would be possible in principle but impractical. A universal Turing machine U accepts both an encoded machine $\langle M \rangle$ and an input x :

$$U(\langle M \rangle, x) = M(x).$$

It reads the description of M and simulates M step by step on x . Thus, one fixed machine can perform any computation described by another Turing machine.

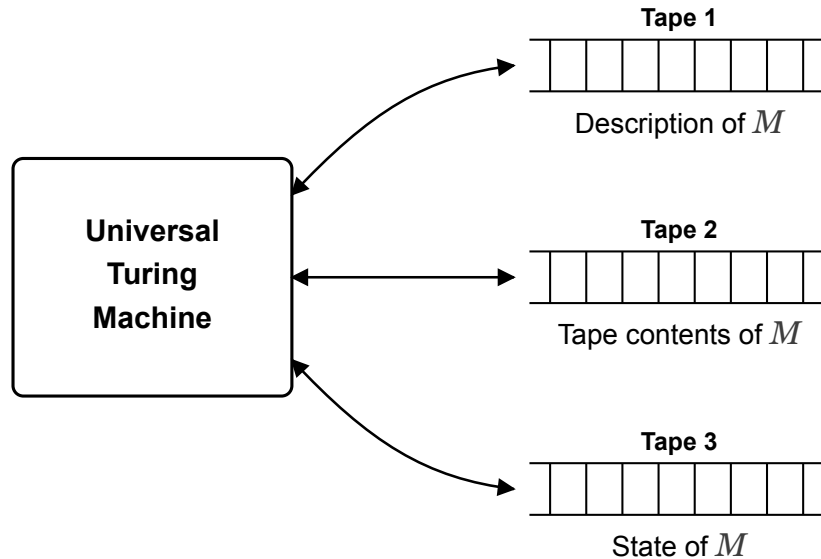


Figure 6: A three-tape organization of a universal Turing machine.

In this organization, Tape 1 stores the encoded description of M , Tape 2 stores the simulated contents of M 's tape, and Tape 3 records M 's current state. The universal machine repeatedly consults the description on Tape 1 and updates Tapes 2 and 3 exactly as M would, thereby reproducing M 's computation one transition at a time.

The key idea is that a program and ordinary data can both be represented as symbols. A program can therefore be stored, copied, inspected, interpreted, compiled, or generated like other data. This *program-as-data* principle underlies stored-program computers, operating systems, compilers, interpreters, program analyzers, and coding tools.

5 The Church–Turing Thesis

Before Turing, people had an intuitive idea of computation: if a task can be broken into clear, fixed steps, then a trained person can solve it simply by following those steps. Long division and the Euclidean algorithm are examples. The person may need time, paper, and practice, but no new creative idea is needed during the calculation. The important question is whether this intuitive idea of a mechanical procedure can be captured by one precise mathematical model.

Church–Turing thesis

Every function that is effectively computable by an algorithm can be computed by a Turing machine.

The Church–Turing thesis says that this intuitive kind of mechanical calculation and Turing-machine computation have the same power. If a human can calculate something by faithfully following finitely described rules, a Turing machine can carry out an equivalent procedure. Conversely, every Turing-machine computation is itself a sequence of mechanical steps. The thesis therefore connects our informal intuition with a formal model and gives us a precise way to use the word *computable*.

Computability is only about whether some algorithm exists. It does not say that the algorithm is fast, memory-efficient, or practical. A computation that takes billions of years is still computable in this sense.

5.1 Why it is called a thesis

A *thesis* and a scientific *theory* play different roles; they are not simply earlier and later stages of the same thing. A scientific theory, such as the Big Bang theory, evolution by natural selection, or the curvature of spacetime in general relativity, explains empirical phenomena and is evaluated against observations. The Church–Turing thesis instead connects an informal human concept—what can be calculated mechanically—with a formal mathematical model. Evidence can make us highly confident in this connection, but the connection does not change into an empirical explanatory theory. That is why it is not expected to “graduate” from a thesis into a theory.

5.2 Support for the thesis

Strong support for the thesis comes from many independently proposed models of computation—including lambda calculus, register machines, general-purpose programming languages, and idealized computers with unbounded memory. Although these models look different, they can simulate one another and compute the same class of functions. Many algorithms developed before and after Turing—such as long division, the Euclidean algorithm, and integer multiplication—also fit within this class. No accepted example of an effective mechanical procedure has been shown to compute something beyond it.

Alonzo Church approached the question through lambda calculus, while Alan Turing approached it through machines modeled on a human calculator. The two formalisms were shown to have the same computational power. Their independent convergence is reflected in the name *Church–Turing thesis*.

This agreement does not prove the thesis from its informal side, but it explains why the thesis is so widely accepted. Once a problem is known to be computable, we can then ask the separate question of how efficiently it can be solved.

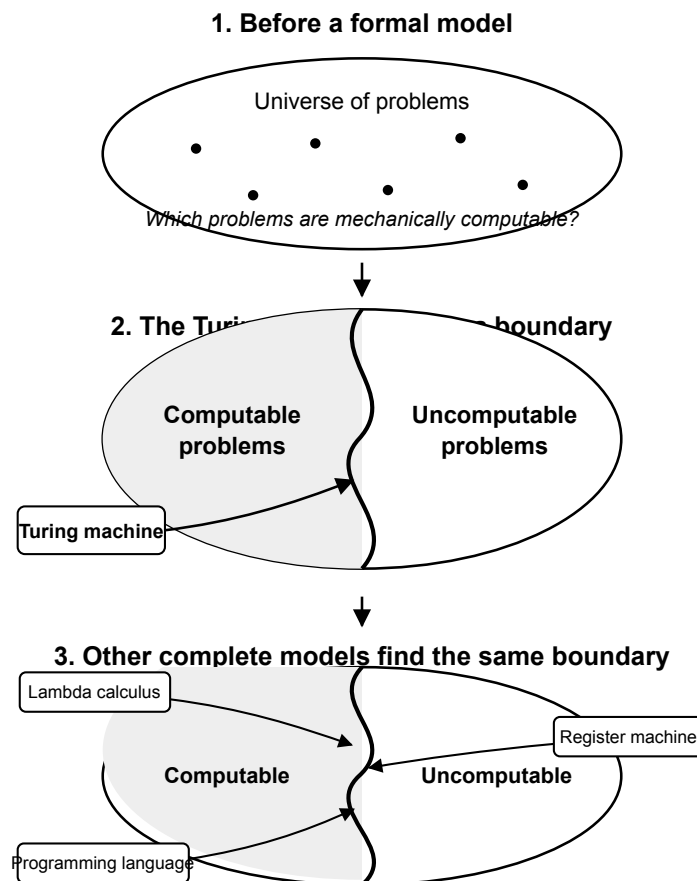


Figure 7: Formal models of computation identify the same boundary between computable and uncomputable problems. Before the boundary is formalized, the problem universe is not yet classified; the Turing machine makes the division precise, and other Turing-complete models recover the same division.

5.3 Turing completeness

A system is called *Turing complete* if it can simulate a universal Turing machine, assuming that memory and execution time may grow without a fixed bound. Intuitively, such a system needs expandable memory, a way to choose among different actions, and a way to repeat computation without a fixed bound. These abilities may be expressed through arrays or maps, conditional branches, loops, recursion, jumps, or equivalent mechanisms; they need not look exactly like a Turing machine's tape and head.

For example, an idealized Python implementation with unbounded memory is Turing complete. HTML by itself is a markup language rather than a general-purpose programming language: it describes document structure but does not provide the unbounded conditional computation required here. Real computers have finite memory, but Turing completeness concerns the language or machine model under the same idealization of memory used for a Turing machine.

To establish that two models have equal computational power, we can show how each simulates the other: computations in one model are translated into computations in the other while preserving their results. Turing machines, lambda calculus, register machines, and general-purpose programming languages look different, but these mutual simulations place them on the same computability boundary. Turing completeness describes what a system can compute in principle, not whether a particular program will halt or run efficiently.

5.4 What would challenge the thesis?

A genuine counterexample would be an effective mechanical procedure that computes a function no Turing machine can compute. Proposed “hypercomputers” often depend on assumptions such as exact infinite precision, infinitely many steps in finite time, or exotic space-time structures. These proposals challenge what physical processes might be possible rather than providing an accepted counterexample. Ordinary quantum computers do not overturn the thesis: they may compute some results more efficiently, but standard quantum computation remains within the class of Turing-computable functions.

6 Limits of Computation

The Church–Turing thesis tells us what counts as mechanical computation. It does not say that every clearly stated problem can be solved mechanically. We now move from defining computation to locating its limits. Computable numbers first give a broad counting argument that such limits must exist; the halting problem then gives a specific problem and proves directly that no algorithm can solve it in full generality.

6.1 Computable numbers

Turing began with *computable numbers* because producing digits is a simple, concrete example of computation. To ask whether a number is computable is to ask whether one finite mechanical procedure can produce as many of its digits as requested. By modeling an idealized human calculator, Turing made the informal phrase “mechanically calculable” mathematically precise. His point was broader than decimal expansions: the same machine model can perform any rule-based symbolic calculation.

A real number is computable if a finite program can generate approximations to it to any requested precision. Every integer and rational number is computable: a rational number is specified by a finite numerator and denominator, and division can produce its decimal digits, including a repeating expansion. The rational numbers are countable: we can arrange numerator–denominator pairs on a two-dimensional grid and enumerate the grid as a one-dimensional sequence of natural numbers, skipping duplicate representations. This is the same basic idea used earlier to map a two-dimensional sheet of cells onto a one-dimensional Turing-machine tape.

The natural numbers and rational numbers therefore have the same cardinality, written $\aleph_0$, even though the rationals appear more densely packed. The real numbers have cardinality $2^{\aleph_0}$ and cannot be placed in a one-to-one correspondence with the natural numbers. Even the interval $[0, 1]$ has the same cardinality as the entire real line. Infinity is thus not a single size: the uncountable infinity of the reals is strictly larger than the countable infinity of finite programs.

Optional set-theory note

The continuum hypothesis asks whether $2^{\aleph_0} = \aleph_1$. It is not needed for the computability argument here: countable programs already cannot cover an uncountable set of real numbers.

Some irrational numbers are computable too. For example, the Leibniz series

$$\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots \right)$$

provides a rule for approximating π , although it converges slowly. Other algorithms can compute its digits much faster. Computability says that a finite rule exists; it does not promise a practical running time.

There are only countably many finite programs, so there are at most countably many computable real numbers. The real numbers are uncountable; therefore, the computable reals form a measure-zero subset of the real line. This is not a meaningful percentage of all “real-world problems”: unlike the reals, that informal collection has no specified measure. The limitation is not that the tape cannot hold a long output—the tape is unbounded—but that a finite program cannot specify a digit-generating rule for every arbitrary real number. Thus, the counting argument proves that noncomputable objects exist, but it does not yet give a concrete, ordinary-looking decision problem that we can identify and prove unsolvable. The halting problem supplies that next step.

6.2 The halting problem

The halting problem is a decision problem: given a program P and input x , answer *yes* if $P(x)$ eventually halts and *no* if it runs forever. At first this may seem like a problem that a sufficiently clever analyzer should solve. We will show, however, that no single algorithm can answer it correctly for every possible program and input.

The proof uses two ideas that also appear in familiar paradoxes: *self-reference* and *diagonalization*. The liar sentence—“This sentence is false”—refers to itself and contradicts either truth value assigned to it. In the barber paradox, a barber shaves exactly those people who do not shave themselves; asking whether the barber shaves himself makes either answer contradict the rule. Self-reference makes an object, rule, or program act on its own description. Diagonalization focuses on the cases in which each object is paired with itself and constructs a new case that gives the opposite answer. The halting proof turns this pattern into a precise argument about programs rather than a linguistic puzzle.

Suppose, for contradiction, that a universal procedure $H(P, x)$ always decides correctly whether $P(x)$ halts. Construct a program D that asks H about a program running on its own description and then deliberately does the opposite of the prediction:

```
def D(P):  
    if H(P, P):  
        loop_forever()  
    else:  
        return
```

Now run $D(D)$. If $H(D, D)$ predicts that this run halts, $D(D)$ loops; if it predicts that the run loops, $D(D)$ halts. Either answer contradicts the prediction. This combination of *self-reference* and a reversal of the predicted behavior is a diagonal argument. Therefore, no such universal halting decider exists.

Blackboard illustration placeholder

Classroom halting-problem diagram: a hypothetical decider $H(P, x)$ inside a program $D(P)$; D loops when $H(P, P)$ says “halts” and returns when it says “does not halt.” Feed D its own description to show the contradiction.

Replace this box with the classroom image.

Self-reference alone is not a contradiction: ordinary recursion, such as the Fibonacci recurrence, also refers to itself. The contradiction arises because D reverses a prediction about its own behavior. This construction is possible because programs can be represented as data—the same principle that enables a universal Turing machine, compiler, or interpreter to process another program.

Reduction: from halting to zero output

A *reduction* solves one problem by translating it into another. Suppose there were a procedure $Z(Q, y)$ that always decided whether program Q on input y eventually returns zero. Given an

arbitrary halting instance (P, x) , construct the following program:

```
def Q(y):  
    P(x)  
    return 0
```

The constructed program $Q(y)$ ignores y and returns zero exactly when $P(x)$ halts. Therefore, if Z existed, running $Z(Q, y)$ for any y would decide the halting problem. Since no halting decider exists, no universal zero-output decider exists either. We write this relationship informally as

$$\text{HALTING} \leq \text{RETURNS-ZERO}.$$

The same strategy transfers known impossibility or difficulty from one problem to another and appears throughout computability and complexity theory.

From one reduction to Rice's theorem

Returning zero is only one property of program behavior. Rice's theorem extends the lesson to every nontrivial semantic property of the function computed by an arbitrary program: if some programs have the property and others do not, no single analyzer can decide that property correctly for all programs.

The price of universality

Universal computers can simulate arbitrary programs, but for every nontrivial semantic property, no program can decide correctly in all cases whether an arbitrary program has that property.

This result does not make program analysis useless. Static analyzers can still find many bugs and security issues by restricting the programs they analyze or by allowing false alarms and missed cases. Rice's theorem concerns what a program computes; it does not automatically apply to every syntactic property or to every question about resource usage.

Expressiveness versus decidability

The more behavior a system can express, the more difficult it becomes to decide general facts about every object in that system. Universality gives a model the power to represent arbitrary programs, including programs that consume their own descriptions; the price is that some behavioral questions become undecidable.

A restricted system may be easier to analyze precisely because it rules out many behaviors. A highly expressive programming language admits far more useful programs, but no analyzer can settle every nontrivial semantic question about all of them. This is not a defect unique to Turing machines; any model capable of universal computation inherits equivalent limits.

6.3 Computation, cognition, and consciousness

Having established formal limits on algorithms, we can now ask what these results do—and do not—tell us about the human mind. Such questions must first be stated precisely. *Computability* asks whether a task can be carried out by a mechanical procedure. To study a task in this way, we must specify its inputs, required outputs, and the conditions under which an answer is correct.

Many *cognitive* abilities can be described as information-processing tasks. Calculating, recognizing patterns, remembering, planning, and using language can therefore be studied with computational models. This does not establish that every aspect of human cognition is computable, but it allows us to ask precise questions about representations, procedures, and observable behavior.

Consciousness raises a different question. It concerns subjective experience: what seeing a color, feeling pain, or being aware feels like from the inside. A system may reproduce the observable input–output behavior of a cognitive task, but that achievement alone does not tell us whether the system has subjective experience. Computability concerns what a procedure can do; consciousness concerns what, if anything, the process feels like.

The Church–Turing thesis characterizes mechanical computation, and undecidability results identify limits on what algorithms can determine. Neither claim establishes that the human mind is entirely computational, and neither shows that consciousness lies beyond computation. Those broader questions require both clearer concepts and empirical investigation.

7 Stored-Program Computers

The von Neumann architecture is a practical organization for an electronic stored-program computer. Its CPU contains a control unit, an arithmetic and logic unit (ALU), and a small collection of registers. One special register, the *program counter*, holds the address of the next instruction. Main memory stores both instructions and data, so programs themselves can be loaded, copied, and modified as data.

The CPU repeatedly performs a fetch–decode–execute cycle:

1. fetch the instruction whose address is in the program counter;
2. decode the instruction;
3. execute it and update registers or memory;
4. update the program counter.

Conditional branches and jumps change the program counter, making selection and repetition possible.

A Turing-machine tape is accessed by moving one cell at a time. Modern main memory instead behaves as random-access memory: a processor supplies an address and accesses that location directly. A finite address width places a physical limit on real memory, even though theoretical analysis often treats memory as expandable. Electronic computers also specialize the tape alphabet to binary symbols represented physically by circuits.

The Turing machine is therefore a mathematical model of what can be computed, whereas the von Neumann architecture is an engineering organization for building electronic computers. Both combine finite control with writable memory, but the architecture adds practical structures—addressed memory, an ALU, registers, buses, and input/output—to execute instructions efficiently. Storing code and data in shared memory enables general-purpose computing, while also requiring protection mechanisms so that data cannot be mistaken for or used to corrupt executable instructions.

Blackboard illustration placeholder

Von Neumann architecture: CPU with control unit, ALU, registers and program counter; shared memory containing code and data; input/output connections.

Replace this box with the classroom image.

8 Complexity: The Cost of Computation

Computability asks whether some algorithm exists. Complexity asks how resources grow with input size. The most common resources are running time and memory.

8.1 Input size and asymptotic growth

For an integer, input size usually means the number of digits or bits, not the integer's numerical value. If a decimal integer has n digits, a digit-by-digit algorithm naturally performs work as a function of n . At the machine level we normally measure the binary representation, so an integer N requires about $\log_2 N$ bits. Complexity statements are meaningful only after the input size has been identified.

Big- O notation describes an asymptotic upper bound. Formally, $f(n) = O(g(n))$ if there are constants $c > 0$ and n_0 such that

$$0 \leq f(n) \leq c g(n) \quad \text{for every } n \geq n_0.$$

It suppresses constant factors, lower-order terms, exact processor speed, and behavior on small inputs. For example,

$$n^2 + n + 1 = O(n^2), \quad 2^n + n^3 + n^2 = O(2^n).$$

Common growth rates include

$$O(1), \quad O(\log n), \quad O(n), \quad O(n \log n), \quad O(n^2), \quad O(n^3), \quad O(2^n), \quad O(n!).$$

For sufficiently large n , the shape of the growth function matters more than small constant differences. Polynomial-time bounds such as $O(n)$, $O(n^2)$, and $O(n^k)$ are treated as efficient in complexity theory. Exponential and factorial growth quickly become infeasible as n increases. In practice, even a polynomial algorithm may be too costly when its exponent or constant factors are large, so linear or near-linear algorithms are often preferred.

Blackboard illustration placeholder

Growth-rate curves for $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, and $O(n!)$, showing the polynomial-time region and the rapid growth beyond it.

Replace this box with the classroom image.

8.2 Searching an attendance list

Finding a name in an unsorted attendance list by checking entries in order takes $O(n)$ time and $O(1)$ extra space. If the list is sorted, binary search takes $O(\log n)$ comparisons: about 7 for roughly 100 entries, 20 for one million, and 30 for one billion. Binary search depends on the list already being sorted.

A hash table records where values are stored. After preprocessing, it provides $O(1)$ expected lookup time while using $O(n)$ additional space. Remembering the location of your own name resembles one constant-time lookup; a general algorithm must store locations for all names if it is to answer arbitrary queries. This example illustrates both the importance of input representation and the recurring time–space tradeoff.

8.3 Fibonacci numbers

The recurrence

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2)$$

does not determine a unique algorithm.

Method	Time	Extra space	Main idea
Naive recursion	$O(2^n)$	$O(n)$	Recomputes the same subproblems
Dynamic programming	$O(n)$	$O(n)$	Stores all previously computed values
Iterative	$O(n)$	$O(1)$	Keeps only the two latest values

Naive recursion follows the mathematical recurrence directly, but its call tree recomputes values such as $F(n-2)$ many times. Only one root-to-leaf chain must remain active at a time, so the recursion uses $O(n)$ stack space even though the total number of calls is exponential. Dynamic programming eliminates the repetition by storing a table of earlier values. Iteration observes that the next value depends only on the previous two, so it keeps two variables and reduces the extra space to $O(1)$. A recursive definition therefore does not force us to use a recursive algorithm.

Blackboard illustration placeholder

Recursion tree for naive Fibonacci, highlighting repeated calls such as $F(n-2)$.

Replace this box with the classroom image.

A modern example: attention and long inputs

The same growth-rate reasoning appears in modern machine learning. Standard full self-attention compares each of n tokens with every other token, giving a quadratic $O(n^2)$ interaction pattern.

Very long inputs can therefore require substantial time and memory. Architectures that restrict, summarize, or reuse earlier information may approach linear processing, but they change what information is retained or compared directly. The simpler search and Fibonacci examples give us the vocabulary needed to recognize this practical tradeoff.

8.4 P and NP

P and NP

P is the class of decision problems solvable in polynomial time. **NP** is the class of decision problems for which a proposed yes-certificate can be verified in polynomial time.

The letters NP stand for *nondeterministic polynomial time*, not “non-polynomial.” The certificate definition above is the most accessible equivalent view: a yes-answer may be difficult to discover, but once a suitable witness is supplied, a deterministic algorithm can check it efficiently.

Every problem in P is in NP because a problem that can be solved efficiently can also have its answer verified efficiently. Whether every NP problem is also in P remains unknown:

$$P \stackrel{?}{=} NP.$$

Subset Sum illustrates the distinction. Given integers $s_1, \dots, s_n$ and a target T , the decision problem asks whether there are choices $b_i \in \{0, 1\}$ such that

$$\sum_{i=1}^n b_i s_i = T.$$

A proposed set of choices can be checked in $O(n)$ arithmetic operations, but naive search examines as many as 2^n subsets. The exponential brute-force algorithm does not prove that no polynomial-time algorithm exists; that missing proof is precisely part of the P-versus-NP difficulty. Most computer scientists believe that $P \neq NP$, reflecting the intuition that finding a solution can be fundamentally harder than checking one, but this belief is not a proof.

Blackboard illustration placeholder

Complexity-class diagram inside the computable region: $P \subseteq NP$, with two possible pictures for $P = NP$ and $P \neq NP$; place exponential-time algorithms outside the polynomial-time region without implying that a problem with a known exponential algorithm cannot also have a better algorithm.

Replace this box with the classroom image.

8.5 The complexity zoo

Complexity theory groups problems by the resources required to solve them and by the computational models allowed. P and NP are only two members of a much larger “complexity zoo.” Other classes capture logarithmic space, randomized computation, parallel computation, counting, interactive proofs, and many other resource restrictions. Some containments and separations are known, while others remain open.

The reduction method introduced with the halting problem also organizes this zoo. Here the translation itself must be efficient: if every instance of problem A can be translated in polynomial time into an instance of problem B , then an efficient algorithm for B would also give an efficient algorithm for A . Instead of analyzing every problem from scratch, computer scientists compare a new problem with problems whose complexity is already understood.

9 Hierarchical Abstraction

The Turing machine tells us what a mechanical computer can compute. Computer architecture explains how a physical machine executes instructions. Abstraction explains how people build usable systems on top of that machinery.

Abstraction and interface

An *abstraction* hides details that are unnecessary for the current task. An *interface* specifies what a component makes available to its users without requiring them to know its internal implementation.

When writing `2 + 3` in Python, a programmer does not need to reason about transistors. The operation passes through layers such as language semantics, interpreter or compiler, machine instructions, digital circuits, logic gates, transistors, and electrical behavior. Each layer exposes a simpler interface to the layer above it.

Likewise, clicking *Send* in a messaging application may involve the application interface, operating-system services, network protocols, device drivers, hardware, and physical signals. No single application developer needs to rebuild all of these layers.

Blackboard illustration placeholder

Hierarchical abstraction stack, for example: user → application → operating system
→ network or instruction-set layer → hardware → physical signals.

Replace this box with the classroom image.

9.1 Leaky abstractions

An abstraction leaks when lower-level details affect behavior visible at a higher level. Examples include:

- a Python list appears able to grow, but physical memory remains finite;
- a network request resembles a function call, but latency and failure remain possible;
- a file appears to be a stable sequence of bytes, but storage devices can fail;
- two algorithms implement the same function, but their running times may differ dramatically.

Abstraction does not erase lower layers. It allows us to ignore them until performance, correctness, security, or failure forces us to cross an interface and inspect the implementation.

10 Concept Checklist

You should be able to explain each pair or relationship

- input, output, instruction, algorithm, and function;
- decision, search, counting, generation, and optimization problems;
- model, simulation, mechanical procedure, and computability;
- tape, state, transition function, and halting state;
- Turing machine, universal Turing machine, and Turing completeness;
- stored-program computer, von Neumann architecture, and program counter;
- thesis versus theorem, and the Church–Turing thesis;
- cardinality, countability, and computable numbers;
- computability versus decidability, including the halting problem and reduction;
- expressiveness versus decidability;
- input size, Big- O , time complexity, and space complexity;
- recursion, dynamic programming, and iterative computation;
- P, NP, efficient solution, efficient verification, and the complexity zoo;
- abstraction, interface, implementation, and leaky abstraction.

11 Study Questions

1. Why does the long-division example qualify as a mechanical procedure?
2. What changes when divisibility is posed as a decision problem rather than a remainder-search problem?
3. Why does a Turing machine use an unbounded tape but only finitely many control states?
4. Using the state-update rule $t = 10r + d$, reproduce the remainder sequence for $35,500,001/113$.
5. Explain why the Euclidean algorithm and prime factorization may compute the same GCD function while having different costs.
6. What information must a universal Turing machine receive to simulate another machine?
7. Why is the Church–Turing statement called a thesis rather than a theorem?
8. How does program-as-data enable both stored-program computing and the halting-problem proof?
9. How does a reduction from halting show that no program can decide whether every arbitrary program returns zero?
10. Why can greater expressiveness make general behavioral questions less decidable?
11. Compare linear search, binary search, and hash lookup. What assumptions and tradeoffs distinguish them?
12. Why do naive recursion and iteration have different costs for Fibonacci numbers even though both compute $F(n)$?
13. State the P versus NP question without saying that NP means “non-polynomial.”
14. Give one example of an abstraction leak and identify the lower-level detail that becomes visible.

Homework

For Homework 2, watch the assigned materials on the following two views of computer science:

- *The Map of Computer Science*, which surveys the major areas and topics studied across the discipline;
- the *Nand to Tetris* overview, which follows the abstraction stack from logic gates and circuits through machine instructions, operating systems, and applications.

Use the first as a horizontal map of the field and the second as a vertical map of how a computer system is built. Review the concept checklist in preparation for a short quiz in the next lecture.

Sources Used for These Notes

These notes synthesize the slide deck *01 Computation and Abstraction.pptx*, its embedded speaker notes, the two classroom transcripts dated 8 September 2026, and the provided classroom transcripts dated 15 and 22 September 2026. The wording has been edited for clarity, and transcription errors have been corrected using the mathematical context. Blackboard drawings that were described in speech but unavailable as images remain as labeled replacement boxes.

For historical context on the Turing machine, see A. M. Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem,” *Proceedings of the London Mathematical Society*, 1936. For a detailed discussion of the Church–Turing thesis, see <https://plato.stanford.edu/entries/church-turing/>.